2026 CSP-S 复赛模考 1 题解

T1 配对序列

【题目大意】

给定一个长度为 n（n 为偶数）的正整数序列 $a_1, a_2, \ldots, a_n$，其中 $1 \le a_i \le m$。

序列被分成如下 $n/2$ 对：

$$(a_1, a_n), (a_2, a_{n-1}), \ldots, (a_{n/2}, a_{n/2+1})。$$

如果存在一个整数 T，使得每一对中两个数的和都等于 T，则称序列是配对的。一次操作可以把任意一个数改成 1 到 m 之间的任意整数，求使序列变为配对的最少操作次数。

【参考难度】

普及+/提高。

【题目解析】

先固定最终的目标和 T。此时每一对只需要独立调整，最后把所有对的代价相加即可。

设当前考虑的一对为 (x, y)。如果只修改其中一个数，那么另一个数不变，因此新的和可以达到的范围为

$$[1 + \min(x, y),\ m + \max(x, y)]。$$

于是这一对在目标和 T 下的最小代价为：

- 若 $x + y = T$，无需修改，代价为 0；
- 否则，若 $1 + \min(x, y) \le T \le m + \max(x, y)$，修改其中一个数即可，代价为 1；
- 否则两个数都需要修改，代价为 2。

目标和一定在 $[2, 2m]$ 内，因此可以枚举 T。

测试点 $1 \sim 4$（40 分）：

$n, m \le 10$。可以枚举目标和 T，再对每一对枚举修改方式。若直接枚举某个被修改的数的新值，复杂度为 $\mathcal{O}(nm^2)$；使用上面的区间结论后，每一对只需 $\mathcal{O}(1)$ 判断，已经可以做到 $\mathcal{O}(nm)$。

测试点 $5 \sim 8$（40 分）：

$n, m \le 100$。继续使用枚举 T、枚举每一对并计算代价的做法，复杂度为 $\mathcal{O}(nm)$，可以通过这一档数据。

测试点 $9 \sim 12$（60 分）：

$n, m \le 5000$。算法不变，仍然枚举 $T \in [2, 2m]$，对每一对使用区间公式计算代价：

```
for (int T = 2; T <= 2 * m; ++T) {
    int res = 0;
    for (int i = 1; i <= n / 2; ++i) {
        int x = a[i], y = a[n - i + 1];
        if (x + y == T) continue;
        int l = 1 + min(x, y), r = m + max(x, y);
        if (l <= T && T <= r) ++res;
        else res += 2;
    }
    ans = min(ans, res);
}
```

时间复杂度为 $\mathcal{O}(nm)$，空间复杂度为 $\mathcal{O}(n)$，期望得分 60 分。

测试点 $13 \sim 14$（70 分）：

特殊性质为所有 a_i 都相等。设所有元素都等于 v，则任意一对都是 (v, v)。令 $T = 2v$，所有对都已经满足要求，答案为 0。

因此可以先用 $\mathcal{O}(n)$ 判断所有元素是否相等；若相等直接输出 0，否则继续使用 $\mathcal{O}(nm)$ 的做法，期望得分 70 分。

正解（100 分）：

$\mathcal{O}(nm)$ 做法的瓶颈是：对于每个目标和 T，都重新扫描全部配对。可以反过来考虑，固定一对 (x, y)，一次性计算它对所有 T 的贡献。

令

$$L = 1 + \min(x, y), \qquad R = m + \max(x, y), \qquad S = x + y。$$

这一对的代价分布为：

- 对 $T \in [L, R]$，代价为 1；
- 对 $T = S$，代价从 1 再减 1，变为 0；
- 其他位置代价为 2。

可以先把所有目标和的代价都设为 2，然后使用差分数组进行区间修改：

```
for (int i = 1; i <= n / 2; ++i) {
    int x = a[i], y = a[n - i + 1];
    int l = 1 + min(x, y);
    int r = m + max(x, y);

    d[2] += 2;
    d[2 * m + 1] -= 2;

    --d[l];
    ++d[r + 1];

    --d[x + y];
    ++d[x + y + 1];
}
```

处理完所有配对后，对差分数组求前缀和，即可得到每个 T 的总操作次数，再取最小值。

每一对只进行常数次差分修改，最后扫描 $[2, 2m]$，因此时间复杂度为 $\mathcal{O}(n + m)$，空间复杂度为 $\mathcal{O}(m)$，期望得分 100 分。

T2 安全接线

【题目大意】

有 n 个接线点，第 i 个接线点的电压为 a_i。当且仅当

$$|a_i - a_j| \le d$$

时，接线点 i, j 之间可以直接接一根线。

对于每次询问 (u, v)，求使 u, v 接通所需的最少线数。如果无论如何都无法接通，输出 -1；特别地，$u = v$ 时答案为 0。

【参考难度】

提高+/省选-。

【题目解析】

直接接线的条件只与电压有关，与接线点原来的编号无关。因此先按电压从小到大排序，记排序后的电压为

$$b_1 \le b_2 \le \cdots \le b_n。$$

对于排序后的位置 i，能与它直接相连的点组成一个连续区间。特别地，如果 $j > i$ 且 $b_j - b_i \le d$，那么位于 i, j 之间的点也一定能与 i 直接相连。

所以图的每个连通块在排序后也是连续的一段。

测试点 $1 \sim 2$（35 分）：

$n, q \le 10$。枚举所有点对，若 $|a_i - a_j| \le d$ 就连边。由于每根线的代价都是 1，可以使用 Floyd 求任意两点之间的最短路：

$$f_{i,j} = \min(f_{i,j}, f_{i,k} + f_{k,j})。$$

时间复杂度为 $\mathcal{O}(n^3)$，每次询问 $\mathcal{O}(1)$，期望得分 35 分。

测试点 $3 \sim 7$（仍为 35 分）：

$n \le 100, q \le 10^4$。算法与上一档相同，Floyd 的复杂度仍为 $\mathcal{O}(n^3)$，可以通过这一档数据。

测试点 $8 \sim 12$（60 分）：

排序后，用双指针求每个位置一步最远可以到达的右端点 r_i：

$$r_i = \max\{j \mid b_j - b_i \le d\}。$$

由于 b 已经有序，可以在线性时间内求出全部 r_i。

接下来求每个位置所在连通块的右端点 `last`。从左到右扫描排序数组：如果当前位置已经超过当前连通块的右端点，就开启新块；否则把当前块的右端点更新为更大的可达位置。查询时先通过 `rk[u]`、`rk[v]` 找到两个原编号在排序数组中的位置，判断它们是否属于同一块；若属于同一块，就从左侧位置开始反复跳到 r_i，直到到达右侧位置。

因为每次都跳到当前能到达的最右位置，所以这种跳法是最优的贪心。单次查询最坏需要 $\mathcal{O}(n)$ 次跳跃，总复杂度为

$$\mathcal{O}(n \log n + qn)\,,$$

期望得分 60 分。

测试点 13（75 分）：

特殊性质 A 为 $d = 0$。此时只有电压相同的接线点之间可以直接连线。相同电压的点构成一个完全图，因此：

- $u = v$ 时答案为 0；
- $u \neq v$ 且 $a_u = a_v$ 时答案为 1；
- $a_u \neq a_v$ 时答案为 -1。

测试点 $14 \sim 15$（75 分）：

特殊性质 B 满足

$$a_1 \le a_2 \le \cdots \le a_n,$$

并且相邻点的电压差不超过 d，隔一个点的电压差超过 d。因此只有相邻位置之间可以直接连边，图退化为一条链，答案就是 $|u - v|$。

正解（100 分）：

60 分做法中，每次查询都要反复执行

$$i \leftarrow r_i。$$

最坏情况下，单次查询需要线性次跳跃。注意到这就是一个典型的倍增过程，可以预处理

$$jump_{k,i} = 从位置\ i\ 连续进行\ 2^k\ 次最远跳后的位置。$$

初值为

$$jump_{0,i} = r_i,$$

递推式为

$$jump_{k,i} = jump_{k-1,jump_{k-1,i}}。$$

查询原编号为 u, v 的两个点时，先令

$$x = rk[u], \qquad y = rk[v],$$

并保证 $x \le y$。如果 $x = y$，答案为 0。

从 x 出发，按照 k 从大到小尝试跳跃：只要跳跃后的位置仍然小于 y，就执行这次跳跃并将答案增加 2^k。循环结束时，当前位置是最后一个小于 y 的位置；如果再跳一步可以到达至少 y，则这一步也计入答案，否则说明两点不在同一个连通块中。

```
int cur = x, ans = 0;
for (int k = LOG - 1; k >= 0; --k) {
    if (jump[k][cur] < y) {
        cur = jump[k][cur];
        ans += (1 << k);
    }
}
if (jump[0][cur] >= y) cout << ans + 1 << '\n';
else cout << -1 << '\n';
```

排序和双指针的复杂度为 $\mathcal{O}(n \log n)$，倍增预处理为 $\mathcal{O}(n \log n)$，每次询问为 $\mathcal{O}(\log n)$，总复杂度为

$$\mathcal{O}((n+q)\log n),$$

空间复杂度为 $\mathcal{O}(n \log n)$，期望得分 100 分。

T3 字符串生成器

【题目大意】

给定长度为 n 的 01 规则串 s 与长度为 m 的模板串 t，构造一个长度为 $n+m-1$ 的小写字符串 w，使得对于每个 $1 \le i \le n$：

- 若 $s_i = 1$，则 $w_i w_{i+1} \dots w_{i+m-1} = t$；
- 若 $s_i = 0$，则 $w_i w_{i+1} \dots w_{i+m-1} \neq t$。

在所有可行的 w 中，输出字典序最小者；如果无解，输出 -1。

下文将 $s_i = 1$ 的位置称为匹配位置，将 $s_i = 0$ 的位置称为不匹配位置。

【参考难度】

提高+/省选-。

【题目解析】

本题可以分成两个阶段：先满足所有必须匹配的窗口，再处理必须不匹配的窗口。

一旦某个位置被匹配窗口锁定，就不能再修改它；其他位置为了让字符串字典序最小，应当先全部填成 `a`。之后，如果某个 `0` 窗口恰好等于 t，就必须修改这个窗口中的某个未锁定位置。为了尽量小地影响字典序，应该选择窗口中最靠右的可修改位置，并将它改成 `b`。

方法一：暴力修改（测试点 $1 \sim 8$，32 分）：

先从左到右处理所有 $s_i = 1$ 的窗口，把 t 填入答案对应的区间。如果某个位置之前已经被填入字符，而这次需要填入的字符不同，则出现矛盾，直接输出 -1。

处理完所有 `1` 后，所有没有被锁定的位置填成 `a`。接下来从左到右处理所有 $s_i = 0$ 的窗口：

1. 暴力比较这个窗口是否等于 t；
2. 如果不等于，则已经满足要求；
3. 如果等于，则从窗口右端向左寻找最靠后的未锁定位置；
4. 将这个位置改成 `b`。

为什么最靠右？因为当前窗口必须被破坏，而越靠右的位置越晚改变，不会影响更早的字典序位置。

$n, m \le 10$ 时，直接枚举修改方式可以通过前 4 个测试点；对于 $n, m \le 5000$，上述算法每次比较窗口和寻找位置都可能花费 $\mathcal{O}(m)$，总复杂度为

$$\mathcal{O}(nm),$$

可以通过测试点 $1 \sim 8$，期望得分 32 分。

这里最重要的不是贪心操作本身，而是证明这个贪心不会破坏已经处理过的条件。

问题一：把待定位置改成 `b`，会不会使更早的某个 `0` 窗口反而变成 t?

不会。假设处理当前位置 i 时，把位置 j 改成了 `b`，但这次修改使更早的某个 0 窗口 $[k, k+m-1]$（$k < i$）变成了 t。

由于 j 是当前窗口中从右向左找到的第一个待定位置，所以 j 右边的相关位置都已经被锁定，且当前位置的修改只能改变某个原本为 `a` 的字符。把模板串按这个位置拆成

$$t = A + \texttt{a} + B + \texttt{b} + C。$$

其中第一个 `a` 对应被修改的位置原来的字符，后面的 `b` 对应当前窗口中更靠右的第一个锁定边界。根据更早窗口与当前窗口的相对位置，也就是比较 $|A|$ 与 $|B|$ 的大小，可以分成三种情况：

- $|A| < |B|$：更早窗口会要求某个位置既等于模板中的 `a`，又等于当前修改后产生的 `b`；
- $|A| = |B|$：两个窗口的对应位置直接矛盾；
- $|A| > |B|$：沿重叠部分继续向左比较，最终仍会得到某个位置既要等于 `a` 又要等于 `b`。

三种情况都矛盾，因此这次修改不可能让更早的 `0` 窗口重新变成 t。

问题二：为什么只需要把字符改成 `b`，而不需要继续尝试 `c`、`d` 等字符?

因为一个待定位置至多会被修改一次。处理某个 `0` 窗口时，选择的是窗口内最靠后的待定位置；修改之后，该位置就不再是待定位置。根据上面对已处理窗口不受影响的证明，后面的 `0` 窗口也不会再次要求修改同一个位置。因此把它改成字典序中紧接着 `a` 的 `b` 就足够了，不需要继续增大字符。

特殊性质（测试点 $9 \sim 16$，64 分）：

- 性质 A：所有 $s_i = 0$。此时没有窗口需要锁定。若 t 不全为 `a`，直接令整个 w 为 `a` 即可，因为每个窗口都已经不等于 t；若 t 全为 `a`，则需要在每个窗口中放置一个非 `a` 字符，可以按间隔选择若干位置改成 `b`。
- 性质 B：所有 $t_j = \texttt{a}$。此时每个 `1` 窗口只要求对应位置为 `a`，不会发生字符冲突，可以用差分数组标记所有被锁定的位置。之后仍可从左到右处理 `0` 窗口并进行贪心破坏。

方法二：Z 函数（测试点 $17 \sim 25$，100 分）：

完整数据中 $n, m \leq 10^6$，不能对每个窗口都进行长度为 m 的比较，因此要把方法一中的两类暴力操作优化掉。

第一类是处理 `1` 窗口之间的重叠冲突。设两个 `1` 窗口的起点相差 `gap`：

- 如果 `gap >= m`，两个窗口没有重叠，不会冲突；
- 如果 `gap < m`，前一个模板的后缀与后一个模板的前缀必须相等，长度为 $m - gap$。

对模板串 t 求 Z 函数后，$z[gap]$ 表示从 t_{gap+1} 开始的后缀与 t 的最长公共前缀长度。因此只需判断

$$z[gap] \geq m - gap$$

即可在 $\mathcal{O}(1)$ 时间判断两个重叠窗口是否冲突。按 `1` 位置从左到右处理时，只填入当前窗口相对之前窗口新增的部分，总填充次数为 $\mathcal{O}(n+m)$。

第二类是判断一个 `0` 窗口是否恰好等于 t。构造

$$t + \# + ans,$$

其中 `#` 是不在小写字母中的分隔字符。对这个字符串求 Z 函数，则从答案位置 i 开始的子串等于 t，当且仅当对应位置的 Z 值至少为 m：

$$z[m + 1 + i] \geq m。$$

最后需要快速找到窗口内最靠后的未锁定位置。可以使用并查集维护每个位置左侧最近的可用位置：删除位置 `pos` 时令它指向 `find(pos-1)`，查询窗口右端点时调用一次 `find`，就能找到窗口内最靠右的可用位置。每个位置最多被删除一次，因此并查集部分的总复杂度为近似线性。

核心流程如下：

```
// 处理 1 位置：用 t 的 Z 函数判断重叠是否冲突
vector<int> z = z_function(t);

// 未锁定位置填 a
for (int i = 0; i < n + m - 1; ++i)
    if (!locked[i]) ans[i] = 'a';

// 用 Z(t + '#' + ans) 判断 0 窗口
string u = t + "#" + ans;
vector<int> z2 = z_function(u);

// 对每个匹配的 0 窗口，找到最靠后的可用位置改成 b
```

时间复杂度为 $\mathcal{O}(n+m)$，空间复杂度为 $\mathcal{O}(n+m)$，期望得分 100 分。

T4 极差之和

【题目大意】

长度为 n 的序列 $a_1, a_2, \ldots, a_n$ 一共有 $n(n+1)/2$ 个不同的连续子段。子段 $[l, r]$ 的极差定义为

$$\mathrm{val}(l, r) = \max(a_l, a_{l+1}, \ldots, a_r) - \min(a_l, a_{l+1}, \ldots, a_r)。$$

从所有连续子段中选出极差最大的 k 个不同子段，求它们的极差之和。

【参考难度】

省选/NOI-。

【题目解析】

测试点 1（12 分）：

$n \le 10, k \le 5$。枚举所有子段，增量维护当前子段的最大值和最小值，得到每个子段的极差。将所有极差排序后取最大的 k 个，复杂度为

$$\mathcal{O}(n^2 \log(n^2))\,,$$

期望得分 12 分。

测试点 $2 \sim 3$（36 分）：

$n \le 100, k \le 100$。仍然可以枚举所有子段并排序，复杂度不变，可以通过这一档数据。

方法一：ST 表 + 最大堆（测试点 $4 \sim 9$，36 分）：

这一档中 $n \le 10^5, k \le 10^5$，不能存储全部 $\mathcal{O}(n^2)$ 个子段。

先观察固定左端点 l 时的情况。随着右端点 r 向右移动，子段包含的元素只会增加，因此最大值不会减小，最小值不会增大，极差不会减小。于是，对每个固定的 l，可以得到一个按 r 排列的单调不减序列：

$$\mathrm{val}(l, l), \mathrm{val}(l, l+1), \ldots, \mathrm{val}(l, n)。$$

如果从右向左取，则它是一个单调不增序列。这样，所有子段的极差就被分成了 n 个有序序列，可以使用最大堆进行多路归并。

ST 表支持在 $\mathcal{O}(1)$ 时间内查询任意区间的最大值和最小值。对每个左端点 l，先将当前序列的第一个元素，也就是区间 $[l, n]$ 的极差放入最大堆。每次取出堆顶，就得到了当前所有未取元素中的最大值；如果该堆顶来自左端点 l，就把这个序列的下一个元素 $[l, n-1]$ 放入堆中，继续归并。

重复 k 次即可得到前 k 大的极差。初始化堆需要 $\mathcal{O}(n)$ 个元素，每次取出和加入元素的代价为 $\mathcal{O}(\log n)$，所以总复杂度为

$$\mathcal{O}((n+k)\log n)\,,$$

空间复杂度为 $\mathcal{O}(n \log n)$，期望得分 36 分。

特殊性质（测试点 $10\sim 16$，64 分）：

性质 A：$a_1 \le a_2 \le \cdots \le a_n$。

此时区间最大值为 a_r，最小值为 a_l，因此

$$\mathrm{val}(l, r) = a_r - a_l。$$

问题变为从所有满足 $l \le r$ 的下标对中选出最大的 k 个差值。可以二分第 k 大的差值 V，用双指针统计差值至少为 V 的下标对数量，并用前缀和求这些差值的总和。

性质 B：$k = n(n+1)/2$，即选择所有子段。

此时不需要考虑排名，只需计算全部子段的极差之和。可以分别统计每个 a_i 作为区间最大值和最小值的次数：

$$\sum_{l \le r} \mathrm{val}(l, r) = \sum_i a_i \times (\text{作为最大值的次数} - \text{作为最小值的次数})。$$

使用单调栈即可在线性时间内求出每个元素左右两侧第一个更大或更小的位置，从而计算贡献。

方法二：二分答案 + 滑动窗口 + 单调队列 + 单调栈 + Lazy 线段树 + 线段树二分（测试点 $17 \sim 25$，100 分）：

当 k 可以达到 10^{12} 时，不能对前 k 个子段逐个取出。改为二分第 k 大极差的值。

设

$$\mathrm{count}(x) = \#\{(l,r) \mid \mathrm{val}(l,r) \geq x\}。$$

因为 x 越大，满足条件的子段越少，所以 `count` 具有单调性，可以二分最大的 V，使得

$$\mathrm{count}(V) \geq k。$$

二分过程中需要高效计算两个量：满足极差至少为 x 的子段数量，以及这些子段的极差总和。

先看数量函数 `count_ge(x)` 。固定右端点 r，维护当前窗口 $[l,r]$ 的最大值和最小值。使用两个单调队列，可以在均摊 $\mathcal{O}(1)$ 时间内得到

$$\max(a_l,\ldots,a_r) - \min(a_l,\ldots,a_r)。$$

当当前窗口的极差至少为 x 时，继续增大左端点，直到窗口极差小于 x。由于极差随着左端点右移不会增大，此时满足条件的左端点恰好是某个前缀区间，数量可以直接累加。每个左端点和右端点最多进出单调队列一次，因此

$$\mathrm{count_ge}(x) = \mathcal{O}(n)。$$

接下来计算 `sum_ge(x)` ，即所有极差至少为 x 的子段的极差之和。固定右端点 r，随着左端点从 r 向左移动，区间最大值只会增加，且只会在遇到新的更大值时发生变化。因此，所有左端点对应的最大值可以被分成若干段，每一段中的最大值相同；最小值同理。

例如固定 r 后，最大值可能形成如下分段：

$$\underbrace{M_1,\ldots,M_1}_{c_1\text{ 个}},\underbrace{M_2,\ldots,M_2}_{c_2\text{ 个}},\ldots$$

单调栈可以维护这些分段。加入新的 a_r 时：

- 对维护最大值的单调栈，栈顶所有不大于 a_r 的段都会与新位置合并；
- 对维护最小值的单调栈，栈顶所有不小于 a_r 的段都会与新位置合并。

于是，每个右端点都能得到左端点从小到大排列时的最大值分段和最小值分段。

网页题解中的实现可以使用 Lazy 线段树：线段树的每个位置代表一个左端点，维护该左端点对应子段的当前最大值或最小值。当右端点向右移动时，单调栈找到受影响的连续左端点区间，并对这些区间进行区间加；同时维护区间和。再结合线段树上的二分，就可以找到满足极差至少为 x 的左端点边界，并求出这些位置上的最大值和最小值之和。

本题标程采用了与上述 Lazy 线段树等价的压缩实现：直接保存单调栈上的值、每段长度以及段长度的前缀和。若要查询前 `lim` 个左端点对应的最大值之和，只需在段长度前缀和上二分找到第 `lim` 个左端点所在的段，再加上完整段贡献和最后一段的部分贡献。

设当前固定右端点为 r，双指针维护出最小的 l，使得所有 $l' \in [0, l-1]$ 的子段 $[l', r]$ 都满足极差至少为 x。于是：

$$\sum_{l'=0}^{l-1} \mathrm{val}(l', r) = \sum_{l'=0}^{l-1} \max(l', r) - \sum_{l'=0}^{l-1} \min(l', r)。$$

分别从最大值分段、最小值分段中求出前 l 个左端点的贡献，累加到 `sum_ge(x)` 中即可。

最后，设 V 是第 k 大极差。所有极差严格大于 V 的子段必须全部选入；极差等于 V 的子段中只需再选足够多个补齐到 k 个。因此答案为

$$\mathrm{sum_ge}(V+1) + V \times \big(k - \mathrm{count_ge}(V+1)\big)。$$

代码中的核心部分为：

```
long long lo = 0, hi = mx - mn;
while (lo < hi) {
    long long mid = (lo + hi + 1) / 2;
    if (count_ge(mid) >= k) lo = mid;
    else hi = mid - 1;
}

long long V = lo;
long long cgt = count_ge(V + 1);
long long sgt = sum_ge(V + 1);
long long ans = sgt + V * (k - cgt);
```

`count_ge` 使用双指针和单调队列，单次复杂度为 $\mathcal{O}(n)$； `sum_ge` 使用单调栈维护分段，并通过前缀和二分或 Lazy 线段树查询贡献，单次复杂度为 $\mathcal{O}(n \log n)$。二分极差值的次数为 $\mathcal{O}(\log V)$，总复杂度为

$$\mathcal{O}(n \log V + n \log n),$$

与 k 的大小无关，空间复杂度为 $\mathcal{O}(n)$ 或 $\mathcal{O}(n \log n)$（取决于 ST 表或线段树实现），期望得分 100 分。